

Taller: API de Reserva de Ambientes de Formación

Tecnologías: Spring Boot · Spring Data JPA · Base de datos relacional (H2 o MySQL)

Modalidad: Individual o en parejas **Trimestre:** Primer trimestre de Spring Boot

Contexto del problema

El centro de formación tiene un problema recurrente: los instructores reservan los ambientes de formación (salas, laboratorios, auditorios) por medio de un grupo de WhatsApp y una hoja de cálculo compartida. El resultado: dos instructores llegan al mismo ambiente a la misma hora, se reservan salas con capacidad insuficiente para el grupo, y nadie sabe qué ambientes están libres en un momento dado.

La coordinación les ha encargado construir **AgendaSENA**, una API REST que centralice la gestión de reservas y **haga cumplir las reglas de uso de los ambientes automáticamente**. La API no debe permitir que se guarde una reserva inválida: las reglas de negocio se validan en el servidor, no se confía en el cliente.

Modelo de datos (mínimo)

Deben modelar al menos estas dos entidades con JPA y la relación entre ellas:

Ambiente

- id, nombre, tipo (SALA, LABORATORIO, AUDITORIO), capacidad (número de personas), activo (boolean)

Reserva

- id, ambiente (relación con Ambiente), nombre del instructor, fecha y hora de inicio, fecha y hora de fin, número de aprendices, estado (ACTIVA, CANCELADA, FINALIZADA)

Pueden agregar una tercera entidad `Instructor` si quieren ir más allá, pero no es obligatorio.

Reglas de negocio (el corazón del taller)

Estas reglas deben implementarse en la **capa de servicio**. Si una regla se incumple, la API responde con un error claro y un código HTTP apropiado (400 o 409 según el caso), nunca con un error 500 ni guardando datos inválidos.

1. **Sin cruces de horario:** un ambiente no puede tener dos reservas ACTIVAS que se solapen en el tiempo, ni siquiera parcialmente. Ejemplo: si existe una reserva de 8:00 a 10:00, no se puede crear otra de 9:00 a 11:00 en el mismo ambiente. (*Pista: piensen bien la condición de solapamiento entre dos intervalos antes de escribir código*).
2. **Capacidad:** el número de aprendices de la reserva no puede superar la capacidad del ambiente.
3. **Horario institucional:** las reservas solo pueden estar entre las 6:00 y las 22:00, y deben durar entre 1 y 4 horas.
4. **Ambientes inactivos:** no se puede reservar un ambiente con `activo = false`.
5. **Límite por instructor:** un instructor no puede tener más de 3 reservas ACTIVAS el mismo día.
6. **Cancelación con anticipación:** una reserva solo puede cancelarse si faltan al menos 2 horas para su inicio. Cancelar no la borra de la base de datos: cambia su estado a CANCELADA.
7. **No se reserva en el pasado:** la fecha de inicio debe ser posterior al momento actual.

Endpoints requeridos

Método	Ruta	Descripción
POST	<code>/api/ambientes</code>	Registrar un ambiente
GET	<code>/api/ambientes</code>	Listar ambientes
POST	<code>/api/reservas</code>	Crear una reserva (aplicando TODAS las reglas)
PATCH	<code>/api/reservas/{id}/cancelar</code>	Cancelar una reserva (regla 6)
GET	<code>/api/ambientes/{id}/reservas?fecha=2026-06-15</code>	Ver las reservas activas de un ambiente en una fecha
GET	<code>/api/ambientes/disponibles?inicio=...&fin=...</code>	Consulta clave: listar los ambientes libres en un rango de tiempo dado
GET	<code>/api/reportes/ocupacion?fecha=...</code>	Reporte: por cada ambiente, cuántas horas estuvo reservado ese día y su porcentaje de ocupación sobre el horario institucional (16 horas)

Noten que esto **no es un CRUD**: no hay update libre de reservas, la cancelación es una operación de negocio, y dos de los endpoints (`/disponibles` y `/ocupacion`) requieren lógica de cálculo, no solo consultar y devolver.

Requisitos técnicos

- Arquitectura en capas: `controller` → `service` → `repository`. La lógica de negocio vive en el servicio, **no** en el controlador.
- Uso de Spring Data JPA con al menos **una consulta personalizada** (query method o `@Query`) para la detección de solapamientos.
- Manejo de errores: respuestas con códigos HTTP correctos y un cuerpo JSON que explique qué regla se incumplió.
- Datos de prueba: cargar al menos 4 ambientes y algunas reservas iniciales (pueden usar `data.sql` o un `CommandLineRunner`).
- Colección de Postman (o archivo `.http`) con los casos de prueba: al menos un caso exitoso y un caso de error **por cada regla de negocio**.

Entregables

1. Repositorio en GitHub con el código y un `README.md` que explique cómo ejecutar el proyecto.
2. Colección de pruebas (Postman o `.http`).
3. Sustentación corta: demostrar en vivo que la regla de solapamiento funciona (intentar crear una reserva que se cruce y mostrar el error).

Criterios de evaluación

Criterio	Peso
Modelo de datos y relación JPA correctos	15%
Reglas de negocio implementadas y validadas en el servicio	35%
Endpoints de disponibilidad y reporte de ocupación funcionando	20%
Manejo de errores con códigos HTTP apropiados	15%
Calidad del código, arquitectura en capas y README	15%

Puntos extra (opcionales)

- Usar DTOs en lugar de exponer las entidades directamente.
 - Validaciones declarativas con Bean Validation (`@NotNull`, `@Min`, etc.) combinadas con las reglas de negocio.
 - Manejo centralizado de excepciones con `@RestControllerAdvice`.
 - Endpoint adicional: "el ambiente más usado de la semana".
-

Sugerencia: antes de escribir una sola línea de código, dibujen en papel dos reservas que se solapan y dos que no. Escriban la condición lógica del solapamiento. Ese es el 35% del taller.